```
list[0..3]  # => [:last, :hash, :step, :exclude_end?]
```

We can check to see whether an object responds to a particular method:

```
r = 1..10
r.respond_to?("frozen?") # => true
r.respond_to?(:key?)     # => false
"me".respond_to?("==")   # => true
```

And we can ask for an object's class and unique object ID and test its relationship to other classes:

```
num = 1
num.object_id            # => 3
num.class                # => Integer
num.kind_of? Integer     # => true
num.kind_of? Numeric     # => true
num.instance_of? Integer # => true
num.instance_of? Numeric # => false
```

Looking at Classes

Knowing about objects is one part of reflection, but to get the whole picture, you also need to be able to look at classes—and the methods and constants that they contain.

Looking at the class hierarchy is easy. You can get the parent of any particular class using superclass, and it's children using subclasses. For classes *and* modules, the ancestors method lists both superclasses and mixed-in modules:

```
klass = Integer
begin
  print klass
  klass = klass.superclass
  print " < " if klass
end while klass
puts
p Integer.ancestors
p Integer.subclasses
```

produces:

```
Integer < Numeric < Object < BasicObject
[Integer, Numeric, Comparable, Object, Kernel, BasicObject]
[]
```

If you want to build a complete class hierarchy, just run that code for every class in the system. We can use ObjectSpace to iterate over all Class objects like this:

```
ObjectSpace.each_object(Class) do |klass|
  # ...
end
```

Looking Inside Classes

We can find out a bit more about the methods and constants in a particular object. We can ask for methods by access level, and we can ask for just singleton methods. We can also take a look at the object's constants, local, and instance variables: